

TextML-Based Pipeline for Web Attack Detection (CIC-IDS2017)

Author: Anas El Mouhtadi

Target Program: Canadian Institute for Cybersecurity Research / Mini-Project Evaluation

Date: September 2026

1. Problem Formulation & Dataset Profiling

Traditional signature-based Network Intrusion Detection Systems (NIDS) struggle against polymorphic payloads and zero-day web exploits. This project implements a Machine Learning (ML) pipeline to detect web-layer attacks (Brute Force, XSS, SQLi) using statistical network flow characteristics.

The pipeline processes the **Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv** subset of the **CIC-IDS2017** dataset. An initial audit revealed a severe class imbalance:

- **BENIGN Traffic:** ~456,797 samples (>99.5%)
- **Web Attacks:** ~2,180 samples (<0.5%)

Relying on standard classification accuracy here leads to the **Accuracy Paradox** (predicting everything as safe yields >99.5% accuracy but misses all attacks). Therefore, this pipeline optimizes for **Precision**, **Recall**, and **F1-Score** on the minority attack class.

2. Data Sanitization & Preprocessing

To ensure numerical stability and prevent data leakage, the following automated steps were executed:

- **Header Normalization:** Stripped leading/trailing whitespace from all 79 feature names.
- **Invalid Value Handling:** Division-by-zero errors in flow-rate calculations generated

`+inf` and `-inf`. These were replaced with `NaN` and purged, leaving 170,231 clean flows.

- **Metadata Purging:** Dropped non-behavioral identifiers (`Flow ID`, `IPs`, `Timestamp`) to prevent "shortcut learning" (memorizing IP addresses instead of attack behaviors).
- **Zero-Variance Elimination:** Dropped constant columns containing identical values across all rows to reduce memory overhead.
- **Binary Mapping:** Encoded `BENIGN` traffic to Class 0 and all web exploits to Class 1.

3. Methodology & Baseline Model

- **Stratified Split & Scaling:** The data was partitioned into an 80% Training Set and 20% Testing Set using a stratified split (`stratify=y`) to maintain the 0.5% attack ratio. A `StandardScaler` was fitted on the training set to prevent test-data leakage.
- **Algorithm Selection:** A **Random Forest Classifier** (100 trees) was selected for its resistance to overfitting and ability to map non-linear feature interactions.
- **Cost-Sensitive Weighting:** To combat the extreme class imbalance, `class_weight='balanced'` was applied. This forces the algorithm to penalize missed attacks heavily during training, adjusting the decision boundaries to prioritize minority class detection.

4. Empirical Results & Evaluation

The model was evaluated on the unseen 20% holdout set (33,611 Benign, 436 Attacks).

Evaluation Metrics:

Traffic Class	Precision	Recall (Detection)	F1-Score	Support
BENIGN (0)	0.9999	0.9999	0.9999	33,611
WEB ATTACK	0.9954	0.9885	0.9919	436

(1)				
Macro Average	0.9976	0.9942	0.9959	34,047

Key Findings:

- **High Interception (98.85% Recall):** The model successfully caught 431 of 436 active attacks, missing only 5.
- **Low Alert Fatigue (99.54% Precision):** Only 2 false alarms were generated across 33,611 legitimate sessions.
- **Feature Importance :** `Init_Win_bytes_forward`, `Flow Duration`, and `Average Packet Size` provided the strongest discriminative power for detecting automated exploits.

5. Discussion, Limitations & Future Work

- **Engineering Challenges:** Standard trees collapsed to the majority class due to the skewed data distribution. Implementing balanced loss weighting resolved this. The model and scaler were also serialized via `joblib` to demonstrate readiness for millisecond-latency live inference.
- **Limitations:** The CIC-IDS2017 dataset reflects a synthetic lab environment; real enterprise networks exhibit higher variance and protocol drift. Furthermore, this flow-based pipeline cannot inspect encrypted Layer-7 payloads (e.g., HTTPS SQLi strings) like Deep Packet Inspection (DPI) can.
- **Future Work:** Next steps include benchmarking Gradient Boosted architectures (LightGBM/XGBoost) for faster line-rate processing, and integrating SHAP (Explainable AI) to give SOC analysts per-flow context on why alerts trigger.

6. Disclosures & Acknowledgments

- **Tech Stack:** Python 3, Scikit-Learn, Pandas, NumPy, Matplotlib, Joblib.
- **Dataset:** Canadian Institute for Cybersecurity (CIC), UNB.
- **Resources & AI Assistance:** Google Gemini was utilized as a technical writing assistant for pipeline scaffolding, edge-case debugging, and report structuring.

Official library documentation (Scikit-Learn, Pandas) and technical community websites were also consulted for implementation references.